

1	2	3	4
5	6	7	8

Exercice 1:

Méthode par inversion (la plus directe): la fonction de répartition est: $F(x) = 1 - \exp(-(\frac{x}{\lambda})^\beta)$, $x \geq 0$.

Posons $U \sim \mathcal{U}_{(0,1)}$.

L'inversion donne: $F^{-1}(u) = \lambda(-\ln(1-u))^{1/\beta}$.

Comme $1-U$ a la même loi que U , on peut utiliser la forme numérique stable: $X = \lambda(-\ln(U))^{1/\beta}$ avec $U \sim \mathcal{U}_{(0,1)}$.

Alors, X suit la loi de Weibull donnée.

Méthode via une exponentielle: Si $Y \sim \text{Exp}(1)$, (densité e^{-y} , $y \geq 0$), alors: $\mathbb{P}(\lambda Y^{1/\beta} \leq x) = \mathbb{P}(Y \leq (\frac{x}{\lambda})^\beta) = 1 - e^{-(x/\lambda)^\beta}$

Donc on a aussi la représentation équivalente: $X = \lambda Y^{1/\beta}$ avec $Y \sim \text{Exp}(1)$.

Pour simuler Y , on peut prendre $Y = -\ln(U)$ avec $U \sim \mathcal{U}(0,1)$, ce qui ramène à la formule par inversion.

Exercice 2:

Transformation par la fonction quantile (méthode d'inversion): Si $U \sim \mathcal{U}_{(0,1)}$, alors $X = \tan(\pi(U - \frac{1}{2}))$ a pour loi de Cauchy standard.

Justification rapide: la fonction de répartition F de Cauchy standard est: $F(x) = \frac{1}{\pi} \arctan(x) + \frac{1}{2}$.

Algorithme (pseudo): Tirer U uniforme sur $(0,1)$

Poser $X = \tan(\pi(U - 1/2))$

X suit la Cauchy standard.

Rapport de deux normales indépendantes: Si N_1 et N_2 sont deux variables normales indépendantes centrées réduites d' $\mathcal{N}(0,1)$, alors: $X = \frac{N_1}{N_2}$ a pour loi de Cauchy standard.

Justification rapide: Passer en coordonnées polaires pour (N_1, N_2) montre que l'angle est uniforme sur $] -\pi, \pi]$ et que le rapport des composantes s'écrit $\tan(\theta)$ avec θ uniforme, d'où la loi Cauchy.

Algorithme (pseudo): Tirer $N_1, N_2 \stackrel{\text{iid}}{\sim} \mathcal{N}(0,1)$

Poser $X = N_1 / N_2$ (retirer si N_2 est strictement proche de 0 si souci numérique)

X suit la Cauchy standard.

Exercice 3:

1) On part de la densité $f(x)$, on calcule sa fonction de répartition $F(x)$, puis on l'inverse.

Pour $x \in [-1, 0]$: $f(x) = 1+x$, donc: $F(x) = \int_{-1}^x (1+t) dt = \frac{(x+1)^2}{2}$

Pour $x \in [0, 1]$: $f(x) = 1-x$, donc: $F(x) = \frac{1}{2} + \int_0^x (1-t) dt = 1 - \frac{(1-x)^2}{2}$.

$$\text{Donc: } F(x) = \begin{cases} \frac{(x+1)^2}{2}, & -1 \leq x \leq 0 \\ 1 - \frac{(1-x)^2}{2}, & 0 \leq x \leq 1. \end{cases}$$

Par simulation X , on tire $U \sim \mathcal{U}_{[0,1]}$ et on résout $F(x) = U$:

- Pour $U \in [0, \frac{1}{2}]$: $\frac{(x+1)^2}{2} = U \Rightarrow x = -1 + \sqrt{2U}$.
- Pour $U \in [\frac{1}{2}, 1]$: $1 - \frac{(1-x)^2}{2} = U \Rightarrow x = 1 - \sqrt{2(1-U)}$

$$\text{Donc: } X = \begin{cases} -1 + \sqrt{2U} & \text{si } U \leq 0,5 \\ 1 - \sqrt{2(1-U)} & \text{si } U > 0,5 \end{cases}$$

2) On sait que $f(x) \leq 1$ sur $[-1, 1]$.

On peut donc choisir comme densité enveloppe $g(x) = \frac{1}{2} \mathbb{1}_{[-1,1]}(x)$ (loi uniforme sur $[-1, 1]$).

$$\text{Alors: } \frac{f(x)}{g(x)} = \frac{(1-|x|)}{\frac{1}{2}} = 2(1-|x|) \leq 2.$$

Donc on peut utiliser $c=2$

Méthode: Générer $Y \sim \mathcal{U}_{[-1,1]}$

Générer $U \sim \mathcal{U}_{[0,1]}$

$$\text{Accepter } Y \text{ si } U \leq \frac{f(Y)}{c g(Y)} = \frac{(1-|Y|)}{1} = 1-|Y|$$

Si non recommencer.

Exercice 4:

1) La densité est: $\alpha \theta^\alpha x^{-\alpha-1} \mathbb{1}_{x \geq \theta}$, $\theta > 0, \alpha > 1$

Fonction de répartition: Pour $x \geq \theta$: $F(x) = \int_0^x \alpha \theta^\alpha t^{-\alpha-1} dt$

$$\text{On trouve: } F(x) = 1 - \left(\frac{\theta}{x}\right)^\alpha, x \geq \theta.$$

$$\text{Donc l'inverse est: } F^{-1}(u) = \theta(1-u)^{-1/\alpha}, u \in]0, 1[$$

Simulation: Générer $U \sim \mathcal{U}_{[0,1]}$

$$\text{Poser } X = \theta(1-U)^{-1/\alpha}.$$

2) Si X a pour densité $f_x(x)$, alors $Y = \frac{1}{x} \Rightarrow x = \frac{1}{y}, dx = -\frac{1}{y^2} dy$.

$$\text{Son support: } x \geq \theta \Leftrightarrow y \leq \frac{1}{\theta}$$

$$\begin{aligned} \text{la densité de } Y \text{ est alors: } f_Y(y) &= f_x\left(\frac{1}{y}\right) \frac{1}{y^2} \\ &= \alpha \theta^\alpha \left(\frac{1}{y}\right)^{-\alpha-1} \frac{1}{y^2} = \alpha \theta^\alpha y^{\alpha-1}, 0 < y \leq \frac{1}{\theta} \end{aligned}$$

$$\text{Donc } Y \text{ suit une loi de type Bêta sur }]0, \frac{1}{\theta}]: f_Y(y) = \alpha \theta^\alpha y^{\alpha-1} \mathbb{1}_{0 < y \leq \frac{1}{\theta}}$$

3) On sait que $Y = \frac{1}{x}$ suit une loi Bêta sur $]0, \frac{1}{\theta}]$.

On peut simuler directement Y : Générer $V \sim \text{Bêta}(\alpha, 1)$ sur $]0, 1[$

$$\text{Poser } Y = \frac{V}{\theta}.$$

$$\text{Puis } X = \frac{1}{Y} = \frac{1}{V/\theta} = \frac{\theta}{V}.$$

Exercice 5:

- 1) Soient $a < b < c < d$ et $\ell = (b-a) + (d-c)$.

La loi uniforme sur $[a, b] \cup [c, d]$ à densité: $f_X(x) = \begin{cases} \frac{1}{\ell} & \text{si } x \in [a, b] \cup [c, d] \\ 0 & \text{sinon} \end{cases}$

La fonction de répartition $F_X(x) = P(X \leq x)$ s'écrit: $F_X(x) = \begin{cases} 0 & \text{si } x < a \\ \frac{x-a}{b-a} & \text{si } x \in [a, b] \\ \frac{b-a}{\ell} & \text{si } x \in]b, c[\\ \frac{b-a}{\ell} + \frac{x-c}{d-c} & \text{si } x \in [c, d] \\ 1 & \text{si } x > d \end{cases}$

Méthode 1: (Inversion/transformation): Soit $U \sim \mathcal{U}_{[0,1]}$

Si $U \leq \frac{b-a}{\ell}$ alors on choisit $X = a + \ell U$ (image dans $[a, b]$)

Sinon (c'est-à-dire $U > \frac{b-a}{\ell}$), on choisit $X = c + \ell(U - \frac{b-a}{\ell})$ (image dans $[c, d]$).

- 3) Description (idée): Choisir une loi candidate g facile à simuler et un c tel que $f(x) \leq c g(x)$ pour tout x .

Ici, une proposition naturelle est $g = \mathcal{U}_{[a, d]}$ (uniforme sur l'ensemble convexe $[a, d]$).

Alors $g(x) = \frac{1}{d-a}$ pour $x \in [a, d]$.

Comme $f(x) = \frac{1}{\ell}$ sur les deux sous-intervalles et 0 ailleurs, on a: $\max_x \frac{f(x)}{g(x)} = \frac{1/\ell}{1/(d-a)} = \frac{d-a}{\ell}$.

On peut prendre $c = \frac{d-a}{\ell}$.

L'algorithme se simplifie beaucoup: tirer $Y \sim \mathcal{U}_{[a, d]}$.

Si $Y \in [a, b] \cup [c, d]$, on accepte Y ; sinon on rejette et on recommence.

Le taux d'acceptation est: $\frac{\ell}{d-a}$.

- 4) On suppose $U_1 \sim \mathcal{U}_{[a, b]}$, $U_2 \sim \mathcal{U}_{[c, d]}$ et $Y \sim \mathcal{B}(p)$ indépendants, avec $p = \frac{b-a}{\ell}$.

Posez $Z = YU_1 + (1-Y)U_2$.

Calculons $F_Z(x) = P(Z \leq x)$, on a alors trois cas:

- Pour $x < a$: $F_Z(x) = 0$.
- Pour $a \leq x \leq b$: $U_2 > x$ toujours (car $U_2 \geq c > b$), donc: $F_Z(x) = P(Y=1)P(U_1 \leq x)$

$$= p \cdot \frac{x-a}{b-a} = \frac{x-a}{\ell}$$
- Pour $b < x < c$: $P(U_1 \leq x) = 1$ et $P(U_2 \leq x) = 0$, donc: $F_Z(x) = P(Y=1) \cdot 1 = \frac{b-a}{\ell}$
- Pour $c \leq x \leq d$: $P(U_1 \leq x) = 1$, $P(U_2 \leq x) = \frac{x-c}{d-c}$.
 Donc: $F_Z(x) = p \cdot 1 + (1-p) \cdot \frac{x-c}{d-c}$

$$= \frac{b-a}{\ell} + \frac{d-c}{\ell} \cdot \frac{x-c}{d-c} = \frac{b-a+x-c}{\ell}$$
- Pour $x > d$: $F_Z(x) = 1$

On retrouve exactement la fonction de répartition de X : Z a la même loi que X .

Méthode de simulation (couplage): Tirer $Y \sim \mathcal{B}(\frac{b-a}{\ell})$

Si $Y = 1$ tirer $U_1 \sim \mathcal{U}_{[a, b]}$ et renvoyer U_1 .

Sinon tirer $U_2 \sim \mathcal{U}_{[c, d]}$ et renvoyer U_2 .

5) Classement recommandé:

1^{er}: Casplage (question 4): Très simple, direct et entièrement vectorisable. Hier une Bernoulli puis la loi uniforme correspondante.

Exact (pas d'approximation), coût constant par tirage.

Très pratique en R (rbinom + runif + ifelse).

2^{ème}: inversion (question 1): Exact et simple à mettre en oeuvre (map linéaire selon U).

Un léger travail de transformation mais aussi entièrement vectorisable.

Avantage: Pas besoin de tirer deux uniformes par point (on tire un seul U).

Comparable en coût au casplage (un tirage uniforme + calcul conditionnel).

3^{ème}: rejet (question 3): Conceptuellement simple mais potentiellement inefficace si l'enveloppe $[a, d]$ est beaucoup plus grande que les intervalles du support (acceptation = $\frac{b-a}{d-a}$).

Ici, l'algorithme se réduit à tirer sur $[a, d]$ et accepter si dans l'union: simple mais variable en coût (répétitions).

Utile si on a déjà un générateur sur $[a, d]$ et qu'on accepte l'overhead; sinon moins bon que les deux autres.

En pratique, pour l'union de deux intervalles disjoints le casplage et l'inversion sont quasiment aussi bons, le casplage est souvent le plus lisible et le plus simple à coder en R.

Le rejet est utile si on préfère garder la notion d'enveloppe unique mais perd en efficacité si le trou (b, c) est large.

Exercice 6:

1) Normalisation: On a $\int_a^b (cx + d) dx = \frac{c}{2}(b^2 - a^2) + d(b - a) = 1$.

Donc: a, b, c et d doivent satisfaire: $\frac{c}{2}(b^2 - a^2) + d(b - a) = 1$.

Non-négativité sur $[a, b]$: Pour tout $x \in [a, b]$, $cx + d \geq 0$.

Comme $cx + d$ est affine, il suffit que sa valeur minimale sur $[a, b]$ soit ≥ 0 , donc:

- Si $c \geq 0$, la valeur minimale est en $x = a$, condition: $ca + d \geq 0$.
- Si $c < 0$, la valeur minimale est en $x = b$, condition: $cb + d \geq 0$.

Résumé: $a < b$, $\frac{c}{2}(b^2 - a^2) + d(b - a) = 1$ et $\min\{ca + d, cb + d\} \geq 0$.

Remarque: Si $c = 0$ alors f est constante sur $[a, b]$ et $d = \frac{1}{b-a}$.

2) Pour $x \in [a, b]$, $F(x) = \int_a^x (ct + d) dt$
 $= \frac{c}{2}(x^2 - a^2) + d(x - a)$.

Donc $F(a) = 0$, $F(b) = 1$.

Méthode d'inversion (algorithme): Pour $U \sim \mathcal{U}_{[0,1]}$, on résout $F(X) = U$ pour $X \in [a, b]$.

- Si $c = 0$: $F(x) = d(x - a)$ donc $x = a + \frac{U}{d} = a + U(b - a)$ (donc uniforme sur $[a, b]$).

- Si $c \neq 0$, écrire l'équation: $\frac{c}{2}(x^2 - a^2) + d(x - a) = U$.

Ceci est une équation quadratique en x .

En la réarrangeant, on a: $\frac{c}{2}x^2 + dx - (\frac{c}{2}a^2 + da + U) = 0$.

Équivalente (multiplier par 2): $cx^2 + 2dx - 2K = 0$, avec $K = \frac{c}{2}a^2 + da + U$.

$$\begin{aligned} \text{Solutions: } x &= \frac{-2d \pm \sqrt{4d^2 + 8cK}}{2c} \\ &= \frac{-d \pm \sqrt{d^2 - 2cK}}{c}. \end{aligned}$$

On prend la racine qui se trouve dans $[a, b]$ (il y en aura exactement une dans l'intervalle).

Ainsi, on simule X par inversion en tirant $U \sim \mathcal{U}_{[0,1]}$ et en calculant X via la formule ci-dessus (ou la version $c = 0$).

- 4) Choisir une densité "candidate" g facile à simuler sur $[a, b]$ (souvent la densité uniforme $g(x) = \frac{1}{b-a}$ sur $[a, b]$).

Trouver M tel que partout $x \in [a, b]$, $f(x) \leq M g(x)$.

Avec g uniforme sur $[a, b]$, $g(x) = \frac{1}{b-a}$, donc $M = (b-a) \cdot \sup_{x \in [a, b]} f(x)$.

Comme $f(x) = cx + d$ est affine, $\sup_{x \in [a, b]} f(x) = \max\{ca + d, cb + d\}$.

Ainsi: $M = (b-a) \cdot \max\{ca + d, cb + d\}$.

Algorithme (rejet): Tirer $Y \sim g$ (ici $Y \sim \mathcal{U}_{[a, b]}$).

Tirer $U \sim \mathcal{U}_{[0,1]}$.

Accepter Y si $U \leq \frac{f(Y)}{M g(Y)}$

↳ Avec $g = \frac{1}{b-a}$, le critère devient $U \leq \frac{f(Y)(b-a)}{M}$

Si accepté, poser $X = Y$; sinon recommencer.

L'efficacité (probabilité d'acceptation) est $\frac{1}{M}$.

A g uniforme, $M \geq 1$ et l'acceptation est souvent acceptable pour densités peu extrêmes.

Exercice 7:

$$\begin{aligned} 1) \text{ Marginalement, pour } y > 0, \text{ on a } f_y(y) &= \int_0^1 f(x, y) dx \\ &= \int_0^1 y x^{y-1} e^{-y} dx \\ &= y e^{-y} \int_0^1 x^{y-1} dx \\ &= y e^{-y} \cdot \frac{1}{y} = e^{-y}. \end{aligned}$$

Donc: $f_y(y) = e^{-y}$ par $y > 0$: $Y \sim \mathcal{E}(1)$ (exponentielle de paramètre 1).

$$\begin{aligned} \text{La densité conditionnelle: } f_{X|Y}(x, y) &= \frac{f_{X,Y}(x, y)}{f_Y(y)} \\ &= \frac{y x^{y-1} e^{-y}}{e^{-y}} = y x^{y-1}, \quad 0 < x < 1. \end{aligned}$$

C'est la densité d'une loi Bêta($y, 1$).

Autre façon de le dire: $X|Y=y \sim \text{Bêta}(y, 1)$.

2) Pour $0 \leq t \leq 1$, $P(X \leq t | Y = y) = \int_0^t y x^{y-1} dx$
 $= [x^y]_0^t = t^y.$

3) Utiliser la décomposition en loi marginale + loi conditionnelle:

1) simuler $Y \sim \mathcal{E}(1)$ (par exemple: $Y = -\ln(U_1)$ avec $U_1 \sim \mathcal{U}_{(0,1)}$):

2) conditionnellement à ce $Y = y$, simuler X selon Bêta $(y, 1)$.

Comme la fonction de répartition est $F_{X|Y}(x) = x^y$, l'inverse donne la simulation par transformée:

$$X = U_2^{1/y}, \text{ où } U_2 \sim \mathcal{U}_{(0,1)} \text{ indépendant.}$$

Exercice 8:

1) $X = (X_1, X_2, X_3) \sim \text{Multinomial}(n, p_1, p_2, p_3)$.

La marge des X_1 est une loi binomiale: $X_1 \sim \text{Binomial}(n, p_1)$, c'est-à-dire, pour $k = 0, 1, \dots, n$,

$$P(X_1 = k) = \binom{n}{k} p_1^k (1 - p_1)^{n-k}$$

$$= \binom{n}{k} p_1^k (p_2 + p_3)^{n-k}$$

2) Pour $X_1 = k$ (avec $0 \leq k \leq n$), on a $(X_2, X_3) | (X_1 = k)$ qui suit une multinomiale de paramètre $n-k$ et probabilités proportionnelles à p_2, p_3 .

Autrement dit, $(X_2, X_3) | (X_1 = k) \sim \text{Multinomial}(n-k, \frac{p_2}{p_2+p_3}, \frac{p_3}{p_2+p_3})$.

En particulier, $X_2 | (X_1 = k) \sim \text{Binomial}(n-k, \frac{p_2}{p_2+p_3})$ et $X_3 = n-k-X_2$.

La probabilité conditionnelle: pour $i+j = n-k$, $P(X_2 = i, X_3 = j | X_1 = k) = \frac{(n-k)!}{i!j!} \cdot (\frac{p_2}{p_2+p_3})^i (\frac{p_3}{p_2+p_3})^j$.

3) 1^{ère} méthode: tirages séquentiels binomiaux: Tirer $x_1 \sim \text{Binomial}(n, p_1)$

Tirer $x_2 \sim \text{Binomial}(n-x_1, \frac{p_2}{p_2+p_3})$

Poser $x_3 = n - x_1 - x_2$

↳ le triplet (x_1, x_2, x_3) a la loi voulue.

2^{ème} méthode: lois catégorielles répétées: Faire n tirages indépendants d'une variable prenant valeur 1, 2 ou

3 avec probabilités p_1, p_2, p_3 , puis compter le nombre d'apparitions de 1, 2, 3.

C'est directement la construction multinomiale.

EXERCICE 1 :

```
# Paramètres de la loi Weibull
lambda <- 2      # paramètre d'échelle
beta <- 1.5      # paramètre de forme
n <- 10000      # taille de l'échantillon
```

```
# Simulation par inversion
u <- runif(n)
x <- lambda * (-log(u))^(1/beta)
```

```
# Histogramme en densité
hist(x,
      breaks = 50,
      freq = FALSE,
      col = "lightblue",
      border = "white",
```

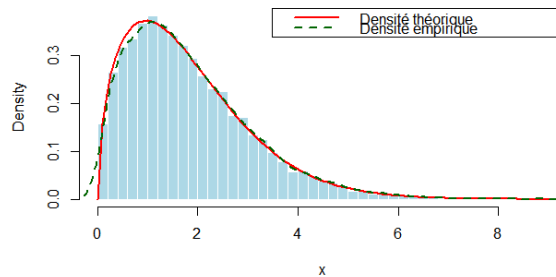
```
main = paste("Loi de Weibull(", lambda, ",", beta,
") - Simulation"),
xlab = "x")
```

```
# Densité théorique
curve(dweibull(x, shape = beta, scale = lambda),
      col = "red", lwd = 2, add = TRUE)
```

```
# Densité empirique (kernel density)
lines(density(x), col = "darkgreen", lwd = 2, lty = 2)
```

```
# Légende
legend("topright",
      legend=c("Densité théorique", "Densité
empirique"),
      col=c("red", "darkgreen"),
      lwd=2, lty=c(1,2))
```

Loi de Weibull(2 , 1.5) - Simulation

**EXERCICE 2 :**

```
# -----
# Simulation d'un échantillon de Cauchy standard
# -----
set.seed(123)      # reproductibilité
n <- 10000         # taille de l'échantillon
x <- rcauchy(n)     # génère n valeurs Cauchy
standard
```

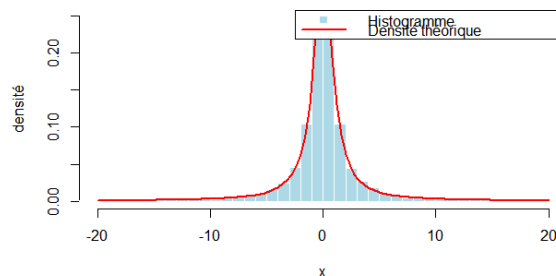
```
# -----
# Histogramme
# -----
hist(x,
      breaks = 5000,
      probability = TRUE, # normalisation en densité
      xlim = c(-20, 20), # zoom sur les valeurs
"raisonnables"
      col = "lightblue",
      border = "white",
      main = "Histogramme d'une Cauchy standard",
```

```
xlab = "x",
ylab = "densité")
```

```
# -----
# Densité théorique superposée
# -----
curve(dcauchy(x), # densité théorique
      from = -20, to = 20,
      add = TRUE,
      col = "red", lwd = 2)
```

```
# -----
# Légende
# -----
legend("topright",
      legend = c("Histogramme", "Densité théorique"),
      col = c("lightblue", "red"),
      lty = c(NA, 1),
      lwd = c(NA, 2),
      pch = c(15, NA))
```

Histogramme d'une Cauchy standard

**EXERCICE 3 :**

```
# --- 1. Méthode par inversion ---
rtriang_inv <- function(n) {
  U <- runif(n)
  X <- ifelse(U <= 0.5,
             -1 + sqrt(2*U),
             1 - sqrt(2*(1-U)))
  return(X)
}
```

```
# --- 2. Méthode par rejet ---
rtriang_rej <- function(n) {
  X <- numeric(n)
  i <- 1
  while(i <= n) {
    Y <- runif(1, -1, 1) # loi enveloppe
    U <- runif(1)        # test
    if(U <= (1 - abs(Y))) {
      X[i] <- Y
    }
  }
}
```

TD 4.8

```

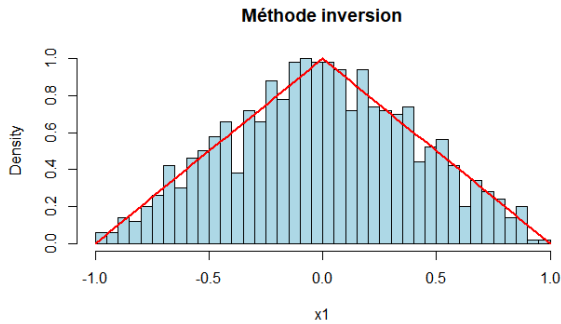
    i <- i + 1
  }
}
return(X)
}

```

```

# Exemple d'utilisation :
set.seed(123)
x1 <- rtriang_inv(1000)
x2 <- rtriang_rej(1000)

```

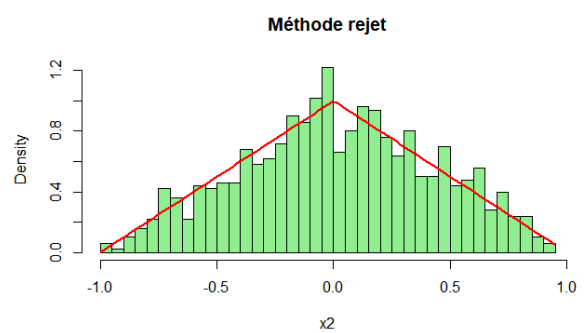


```

# Visualisation
hist(x1, breaks=50, prob=TRUE, col="lightblue",
main="Méthode inversion")
curve((1 - abs(x))*(abs(x)<=1), add=TRUE, col="red",
lwd=2)

hist(x2, breaks=50, prob=TRUE, col="lightgreen",
main="Méthode rejet")
curve((1 - abs(x))*(abs(x)<=1), add=TRUE, col="red",
lwd=2)

```



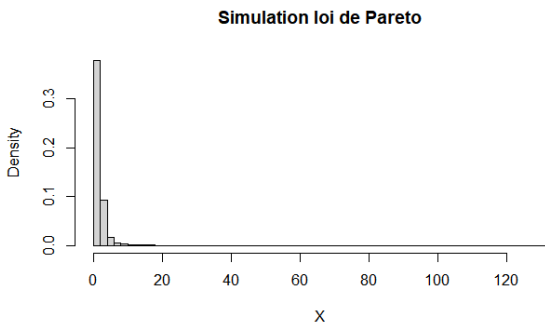
EXERCICE 4 :

```

# Simulation loi de Pareto par inversion
rpareto <- function(n, alpha, theta) {
  U <- runif(n)
  X <- theta * (1 - U)^(-1/alpha)
  return(X)
}

# Exemple :
set.seed(123)
X <- rpareto(10000, alpha = 2, theta = 1)
hist(X, prob = TRUE, breaks = 50, main = "Simulation loi
de Pareto", xlab = "X")

```



```

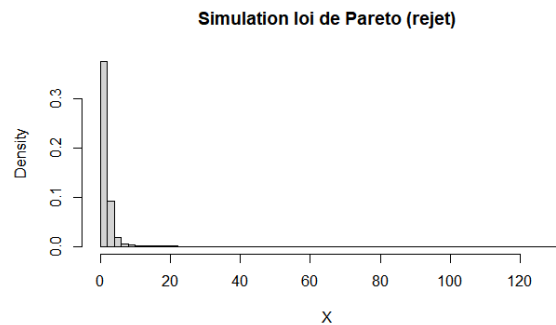
# Simulation loi de Pareto par transformation via Beta
rpareto_rejet <- function(n, alpha, theta) {
  # Génère V ~ Beta(alpha,1)
  V <- rbeta(n, shape1 = alpha, shape2 = 1)
  X <- theta / V
  return(X)
}

```

```

# Exemple :
set.seed(123)
X2 <- rpareto_rejet(10000, alpha = 2, theta = 1)
hist(X2, prob = TRUE, breaks = 50, main = "Simulation
loi de Pareto (rejet)", xlab = "X")

```



EXERCICE 5 :

```

set.seed(123) # pour reproductibilité

n <- 1000
a <- 0; b <- 2; c <- 7; d <- 10
l <- (b-a) + (d-c)

U <- runif(n)
X_inv <- ifelse(U <= (b-a)/l,
  a + l*U,
  c + l*U - (b-a))

# histogramme
hist(X_inv, breaks = 30, probability = TRUE,
main = "Histogramme - uniforme sur [0,2] ∪
[7,10]",
xlab = "x", ylim = c(0, 0.35))
# superposer la densité théorique (constante = 1/l sur
les intervalles)
segments(a, 1/l, b, 1/l, lwd=2) # segment sur [a,b]
segments(c, 1/l, d, 1/l, lwd=2) # segment sur [c,d]

```

```

#rejet
rejection_uniform_union <- function(n, a, b, c, d) {
  out <- numeric(n)
  i <- 1
  while (i <= n) {
    y <- runif(1, min = a, max = d)
    if ((y >= a && y <= b) || (y >= c && y <= d)) {
      out[i] <- y
      i <- i + 1
    }
    # sinon rejet et on recommence
  }
  return(out)
}

# Exemple d'utilisation pour [0,2] ∪ [7,10], n=1000
set.seed(123)
X_rej <- rejection_uniform_union(1000, 0, 2, 7, 10)
hist(X_rej, breaks=30, probability=TRUE,
main="Histogramme (rejet) - uniforme sur [0,2] ∪
[7,10]")

```

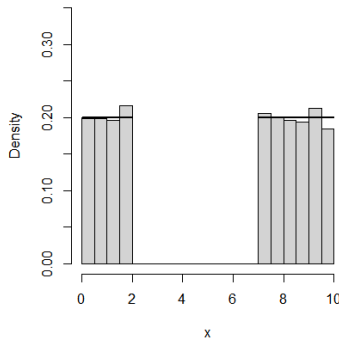
```
# superposer la densité théorique (constante = 1/l sur
les intervalles)
segments(a, 1/l, b, 1/l, lwd=2) # segment sur [a,b]
segments(c, 1/l, d, 1/l, lwd=2) # segment sur [c,d]

#couplage
sim_coupling <- function(n, a, b, c, d) {
  l <- (b-a) + (d-c)
  p <- (b-a)/l
  # tirer Bernoulli p pour chaque observation
  Y <- rbinom(n, size = 1, prob = p)
  # tirer deux vecteurs uniformes
  U1 <- runif(n, min = a, max = b)
  U2 <- runif(n, min = c, max = d)
```

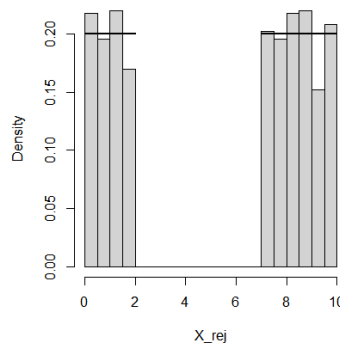
```
X <- ifelse(Y == 1, U1, U2)
return(X)
}

set.seed(123)
X_couple <- sim_coupling(1000, 0, 2, 7, 10)
hist(X_couple, breaks=30, probability=TRUE,
     main="Histogramme (couplage) - uniforme sur [0,2]
     U [7,10]")
# superposer la densité théorique (constante = 1/l sur
les intervalles)
segments(a, 1/l, b, 1/l, lwd=2) # segment sur [a,b]
segments(c, 1/l, d, 1/l, lwd=2) # segment sur [c,d]
```

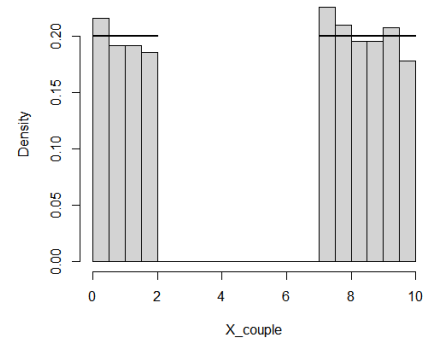
Histogramme - uniforme sur [0,2] ∪ [7,10]



Histogramme (rejet) - uniforme sur [0,2] ∪ [7,1



Histogramme (couplage) - uniforme sur [0,2] ∪ [7,10]



EXERCICE 6 :

```
## Paramètres (exemple)
a <- 0
b <- 1
c <- 1 # modifiez si besoin
# calcul de d par normalisation :
d <- (1 - (c/2)*(b^2 - a^2)) / (b - a)

# vérifications rapides
cat("a,b,c,d =", a,b,c,d, "\n")
cat("min f at endpoints:", c*a + d, ",", c*b + d, "\n")

n <- 1000
U <- runif(n)

# fonction pour inverser F
invert_F <- function(u, a, b, c, d){
  if(abs(c) < 1e-12){ # c == 0 -> uniforme
    return(a + u*(b-a))
  } else {
    K <- (c/2)*a^2 + d*a + u
    discr <- d^2 + 2*c*K
    # discr >= 0 by construction (u in [0,1]) but
    numeric safety:
    discr[discr < 0] <- 0
    x1 <- (-d + sqrt(discr)) / c
    x2 <- (-d - sqrt(discr)) / c
    # choisir la racine qui est dans [a,b]
    x <- ifelse(x1 >= a & x1 <= b, x1, x2)
    # si encore NA, forcer projection (sécurité)
    x[x < a] <- a
    x[x > b] <- b
    return(x)
  }
}

X_inv <- invert_F(U, a, b, c, d)

# quick checks
mean(X_inv)
hist(X_inv, probability=TRUE, breaks=30,
     main="Simulations par inversion")
```

```
# ajouter densité théorique
f_theo <- function(x) ifelse(x>=a & x<=b, c*x + d, 0)
xx <- seq(a, b, length.out=200)
lines(xx, f_theo(xx), lwd=2)

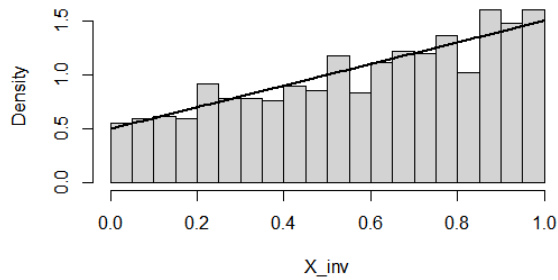
#rejet
# Rejection sampling to get n draws
rejection_sample <- function(n, a, b, c, d){
  # compute M
  f_end_a <- c*a + d
  f_end_b <- c*b + d
  f_sup <- max(f_end_a, f_end_b)
  M <- (b - a) * f_sup
  if(M < 1) warning("M < 1 (unexpected) but continuing")

  res <- numeric(n)
  i <- 0
  iter <- 0
  while(i < n){
    # draw candidate
    y <- runif(1, a, b)
    u <- runif(1)
    f_y <- c*y + d
    if(u <= (f_y * (b-a) / M)){
      i <- i + 1
      res[i] <- y
    }
    iter <- iter + 1
    # (optionnel) protection pour éviter boucle infinie
    if(iter > 1e7) stop("Too many iterations in
    rejection sampling")
  }
  cat("acceptance rate approx:", n/iter, "\n")
  return(res)
}

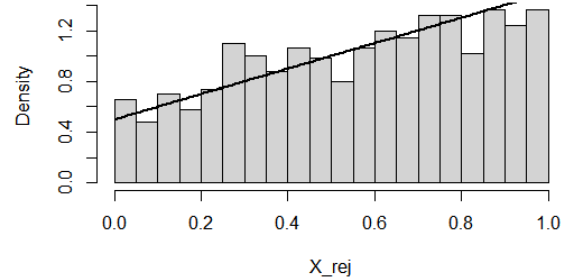
# Exemple: mêmes paramètres que plus haut
X_rej <- rejection_sample(1000, a, b, c, d)

# histogram + densité
hist(X_rej, probability=TRUE, breaks=30,
     main="Simulations par rejet")
lines(xx, f_theo(xx), lwd=2)
```

Simulations par inversion



Simulations par rejet



EXERCICE 7 :

```
# Simulation d'un seul couple (X,Y)
sim_one <- function() {
  u1 <- runif(1)      # Uniforme(0,1)
  y <- -log(u1)        # Y ~ Exp(1)
  u2 <- runif(1)      # Uniforme(0,1)
  x <- u2^(1/y)        # X | Y=y ~ Beta(y,1)
  return(c(X=x, Y=y))
}

# Simulation de n couples (X,Y)
sim_xy <- function(n) {
  u1 <- runif(n)
  y <- -log(u1)        # vecteur de Y
  u2 <- runif(n)
```

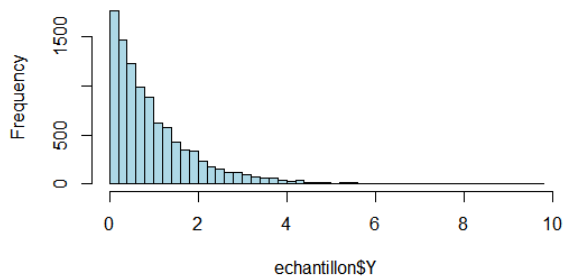
```
  x <- u2^(1/y)        # vecteur de X
  data.frame(X=x, Y=y)
}

# Exemple : simuler 10 000 observations
set.seed(123)
echantillon <- sim_xy(10000)

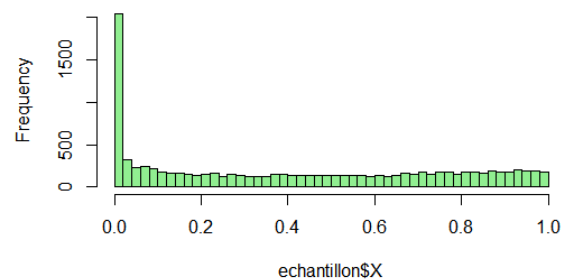
# Aperçu
head(echantillon)

# Histogrammes
hist(echantillon$Y, breaks=50, main="Histogramme de Y
(Exp(1))", col="lightblue")
hist(echantillon$X, breaks=50, main="Histogramme de X",
col="lightgreen")
```

Histogramme de Y (Exp(1))



Histogramme de X



EXERCICE 8 :

```
# Paramètres
n <- 20
p1 <- 0.3
p2 <- 0.5
p3 <- 0.2

# Tirage unique du vecteur (X1,X2,X3)
x <- rmultinom(1, size = n, prob = c(p1, p2, p3))
x # C'est une matrice 3x1 : lignes = catégories,
colonnes = tirages
X1 <- x[1,]
X2 <- x[2,]
X3 <- x[3,]
cat("X1 =", X1, "X2 =", X2, "X3 =", X3, "\n")

# 1000 simulations

> # Moyennes empiriques
> mean_X1 <- mean(X1_vals)
> mean_X2 <- mean(X2_vals)
> mean_X3 <- mean(X3_vals)
> cat("Moyennes :", mean_X1, mean_X2, mean_X3, "\n")
Moyennes : 6.06 10.008 3.932
```

```
sim <- rmultinom(1000, size = n, prob = c(p1, p2, p3))

# Chaque colonne = une simulation, lignes = catégories
X1_vals <- sim[1,]
X2_vals <- sim[2,]
X3_vals <- sim[3,]

# Moyennes empiriques
mean_X1 <- mean(X1_vals)
mean_X2 <- mean(X2_vals)
mean_X3 <- mean(X3_vals)
cat("Moyennes :", mean_X1, mean_X2, mean_X3, "\n")

# Variances empiriques
var_X1 <- var(X1_vals)
var_X2 <- var(X2_vals)
var_X3 <- var(X3_vals)
cat("Variances :", var_X1, var_X2, var_X3, "\n")

> # Variances empiriques
> var_X1 <- var(X1_vals)
> var_X2 <- var(X2_vals)
> var_X3 <- var(X3_vals)
> cat("Variances :", var_X1, var_X2, var_X3, "\n")
Variances : 4.308709 4.922859 3.112488
```